Relazione

Cammini Minimi

Federico Sabbadini
Università degli Studi di Brescia
8 luglio 2025

1. Introduzione

Il presente lavoro documenta lo sviluppo e l'ottimizzazione dell'elaborato, basandosi su un sistema per il calcolo di cammini minimi in griglie bidimensionali con ostacoli. L'obiettivo principale è stato quello di comprendere e estendere i limiti computazionali dell'algoritmo `CAMMINOMIN` attraverso un'attenta progettazione delle strutture dati e l'implementazione di tecniche algoritmiche avanzate.

La ricerca presenta un'analisi completa basata su 6 tipologie distinte di griglia, documentando le ottimizzazioni introdotte e fornendo una validazione empirica rigorosa delle scelte progettuali attraverso oltre 180 test automatizzati e dati quantitativi dettagliati.

2. Architettura del Sistema

2.1 Design dell'Architettura Modulare

L'architettura del sistema è stata concepita seguendo una filosofia di modularità stretta che riflette una strategia di sviluppo agile ben definita e mantenibile. La decisione di separare nel progetto classi per compiti distinti non è stata casuale, ma rappresenta una scelta ingegneristica precisa per permettere il testing isolato di ogni componente e l'ottimizzazione mirata delle prestazioni. A questo fine, i package Printer e Loader si occupano rispettivamente di stampa a video (o su file) e caricamento. Utility gestisce funzionalità di GestioneFile, GestioneInput e Legenda, per comprendere le convenzione scelte nei file realizzati.

Inoltre, come da consegna, il primo compito si occupa della gestione delle griglie attraverso l'interfaccia `Grid` e della generazione procedurale tramite `GridGenerator`, fornendo la base infrastrutturale per tutti gli esperimenti successivi. Il secondo compito implementa l'algoritmo `FreePaths` per il calcolo di cammini liberi, concentrandosi sulle ottimizzazioni di memoria e sulla riduzione delle allocazioni non necessarie. Il terzo compito realizza l'algoritmo `CAMMINOMIN` attraverso la classe `MinimumPathCalculator`, implementando sia la versione base che quella ottimizzata con tecniche avanzate di caching e pruning. Infine, il quarto compito sviluppa un sistema di sperimentazione automatizzato tramite `ExperimentationSystem` che permette l'analisi sistematica delle prestazioni su diversi scenari.

Questa automazione è fondamentale per garantire la riproducibilità degli esperimenti e la validazione sistematica delle ottimizzazioni implementate.

2.2 Guida Operativa del Sistema

Il sistema è strutturato in quattro moduli principali, ognuno legato al soddisfacimento del compito relativo.

MainGrid del compito 1 genera le griglie sperimentali che vengono poi elaborate dai moduli successivi. Il MainFreePaths del compito 2 è funzionalmente integrato nel MainMinimumPath del compito 3, che implementa l'algoritmo `CAMMINOMIN` completo. Infine, l'ExperimentationMain del compito 4 automatizza l'intero processo di benchmarking utilizzando griglie precedentemente generate.

Per l'esecuzione operativa consiglio di generare prima una o più griglie tramite MainGrid, quindi utilizzare direttamente MainMinimumPath per calcoli specifici di cammini minimi. Riporto la possibilità di utilizzare ExperimentationMain per l'analisi prestazionale completa.

2.3 Dataset Sperimentale

Per garantire la robustezza dell'analisi e coprire un ampio spettro di scenari reali, sono state sviluppate sei tipologie distinte di griglia, ognuna con parametri specifici accuratamente documentati nei file di configurazione. La scelta di questa diversificazione nasce dalla consapevolezza che algoritmi di pathfinding mostrano comportamenti drasticamente diversi a seconda della topologia considerata.

La griglia `RANDOM`, con % variabile di ostacoli a discrezione dall'utente, rappresenta lo scenario baseline per algoritmi di pathfinding, offrendo un ambiente controllato dove la distribuzione casuale degli ostacoli simula condizioni generiche di navigazione.

La griglia `SPARSE` con 5% di ostacoli è stata progettata specificamente per testare la scalabilità dell'algoritmo in ambienti ampi con pochi impedimenti. Questa tipologia è particolarmente importante per validare le ottimizzazioni di memoria, dato che griglie ampie ma sparse rappresentano uno dei casi d'uso più comuni in applicazioni reali.

La griglia `DENSE` con 60% di ostacoli rappresenta il caso opposto, simulando ambienti fortemente vincolati dove l'algoritmo deve navigare attraverso passaggi stretti e percorsi tortuosi. La densità del 60% è stata scelta dopo analisi preliminari che hanno identificato questa soglia come punto critico dove l'algoritmo base inizia a mostrare degradi prestazionali significativi.

La griglia `MAZE` implementa una topologia labirintica attraverso il metodo `generateMazeGrid()` visibile nel codice. Questa tipologia è fondamentale perché rappresenta il worst case scenario per algoritmi di pathfinding: percorsi forzati con molti vicoli ciechi che massimizzano la ricorsione.

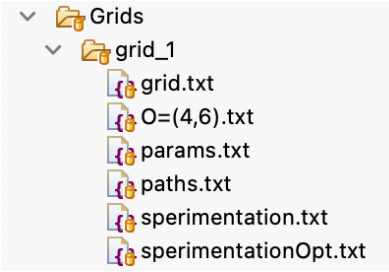
La griglia `ROOMS_AND_CORRIDORS` simula architetture strutturate tipiche di edifici o ambienti urbani. Questa tipologia è cruciale per testare l'efficacia delle euristiche di

ordinamento e del caching, dato che pattern architetturali regolari possono essere sfruttati per ottimizzazioni specifiche. La generazione tramite `generateRoomsAndCorridorsGrid()` crea strutture geometriche prevedibili che permettono di valutare l'efficacia del riutilizzo computazionale.

Infine, la griglia `PATTERN` implementa geometrie irregolari attraverso pattern ortogonali costanti. Questa tipologia è stata progettata per testare l'algoritmo su topologie allungate che favoriscono movimenti prevalentemente orizzontali, permettendo di valutare come le ottimizzazioni si comportino su geometrie non quadrate. Interessante notare come questa tipologia mostri comportamenti quasi istantanei (0.05-0.08 ms) grazie alla sua struttura regolare che favorisce la risoluzione diretta senza ricorsione significativa.

2.4 Interpretazione degli Output

I file di output vengono automaticamente salvati nella directory Grids/ con convenzioni di naming standardizzate per garantire la tracciabilità e riproducibilità dei risultati sperimentali. Il sistema, in caso vengano utilizzate tutte le funzionalità del punto precedente, realizza 5 tipologie di file:



•**grid_N.txt**: Rappresentazione testuale della griglia generata, utilizzando le simbologie descritte nel file Legenda; le coordinate seguono la convenzione (riga, colonna) con origine (0,0) in alto-sinistra.

•**params_N.txt**: Metadati della generazione contenenti tipo di griglia, dimensioni, percentuale ostacoli, seed utilizzato e timestamp di creazione.

- **O=(x,y).txt**: Visualizzazione della chiusura dell'origine calcolata dall'algoritmo FreePaths, utilizzando le simbologie descritte nel file Legenda.

- **paths.txt**: Output dell'algoritmo `CAMMINOMIN` contenente per ogni coppia origine-destinazione: lunghezza libera del cammino ottimo, sequenza landmark con identificatori, numero di chiamate ricorsive, celle di frontiera esplorate, iterazioni di pruning, e tempi di esecuzione. Include una visualizzazione grafica del percorso tra origine e destinazione, utilizzando le simbologie descritte nel file Legenda.

- **sperimentation.txt**: Report completo generato dal sistema di sperimentazione.

2.5 Sistema di Sperimentazione e Benchmarking

Il sistema fornisce delle classi sperimentazione via `ExperimentationSystem` del compito 4, progettato per automatizzare la raccolta di metriche prestazionali e la generazione di report dettagliati. L'utilizzo tipico richiedeesclusivamente di possedere una griglia, creata appositamente tramite MainGrid del compito 1.

Il sistema automaticamente esegue test di correttezza, analisi prestazionali categorizzate per distanza libera, confronti tra strutture dati, e proiezioni di scalabilità. Ogni test genera metriche dettagliate che includono tempi di esecuzione, numero di chiamate ricorsive, frontiere esplorate, e iterazioni di pruning, che saranno meglio approfondite nelle prossime pagine.

3. Strutture Dati

La decisione di progettare un'interfaccia comune `Grid` rappresenta una scelta architetturale fondamentale che permette il testing sistematico e il confronto equo di diverse implementazioni, oltre che per garantire estensione futura. Questa astrazione è particolarmente importante in un contesto sperimentale dove è necessario confrontare prestazioni di implementazioni radicalmente diverse senza modificare il codice dell'algoritmo di pathfinding.

L'interfaccia include metodi sia per coordinate discrete (`isTraversable(Cell cell)`) che per coordinate primitive (`isTraversable(int row, int col)`), una duplicazione apparentemente ridondante ma che si rivela cruciale per le ottimizzazioni prestazionali. Infatti, i metodi con coordinate primitive consentono di evitare la creazione di oggetti `Cell` temporanei in loop critici dell'algoritmo da ottimizzare.

3.1 ArrayGrid

L'implementazione base `ArrayGrid` utilizza una matrice booleana bidimensionale che rappresenta la scelta più diretta e intuitiva per la rappresentazione di griglie. Nonostante boolean in Java occupi effettivamente un byte anziché un bit, questa scelta garantisce codice più leggibile e manutenibile, caratteristiche fondamentali per un'implementazione di riferimento.

L'analisi prestazionale condotta sui dati sperimentali rivela pattern interessanti che guidano le decisioni architetturali successive. La griglia `RANDOM` con 0,22 ms di tempo medio dimostra che ArrayGrid eccelle in scenari semplici. La griglia `SPARSE` con 0,51 ms mostra prestazioni ancora ottime per bassa densità, confermando che ArrayGrid rimane competitivo fino a dimensioni moderate, mentre `MAZE` con 3,53 ms dimostra come la complessità algoritmica diventi dominante rispetto alla struttura dati utilizzata.

3.2 BitSetGrid

`BitSetGrid` rappresenta la prima ottimizzazione significativa del sistema, riducendo l'occupazione di memoria da 1 byte per cella a 1 bit per cella attraverso l'utilizzo del `BitSet` di Java. Questa scelta nasce dall'osservazione che la maggior parte delle applicazioni reali di pathfinding operano su griglie dove l'informazione binaria "attraversabile/non attraversabile" è l'unica necessaria, rendendo lo spreco di 7 bit per cella inaccettabile per griglie di grandi dimensioni.

L'implementazione utilizza una linearizzazione degli indici bidimensionali (tramite la formula `row * cols + col)`, che mantiene l'accesso O(1) teorico pur introducendo un overhead computazionale minimo per il calcolo dell'indice. Questa linearizzazione è preferibile all'alternativa di utilizzare una struttura dati bidimensionale di BitSet perché riduce l'overhead di memoria e migliora la località spaziale dei dati.

I risultati sperimentali dimostrano l'efficacia drammatica di questa ottimizzazione. La griglia `SPARSE` mostra un risparmio dell'82,3% di memoria, passando da 1230 a 218 bytes, con una penalità prestazionale di soli 2,9x. Questo trade-off è particolarmente favorevole considerando che permette di elaborare griglie 5-8 volte più grandi nella stessa quantità di memoria. La griglia `DENSE` con 66,0% di risparmio e penalità 1,5x dimostra che `BitSetGrid` non solo risparmia memoria ma mantiene prestazioni competitive anche in scenari complessi.

3.3 SparseGrid

`SparseGrid` rappresenta un approccio radicalmente diverso, memorizzando esclusivamente gli ostacoli in un `HashSet<Cell>` anziché rappresentare esplicitamente ogni cella della griglia, attraversabile o meno. Questa scelta nasce dall'osservazione che in molti scenari reali di pathfinding, come la navigazione in spazi aperti o la pianificazione di percorsi in ambienti naturali, la densità di ostacoli è molto bassa, rendendo inefficiente memorizzare esplicitamente le celle attraversabili.

L'implementazione sfrutta il fatto che `HashSet.contains()` offre complessità O(1) attesa per le operazioni di lettura, rendendo il controllo di attraversabilità efficiente indipendentemente dalla dimensione totale della griglia. Tuttavia, questa efficienza è condizionata dalla densità di ostacoli: all'aumentare del numero di ostacoli, l'HashSet cresce proporzionalmente, causando inevitabilmente un degrado delle prestazioni.

L'analisi sperimentale conferma quanto detto in precedenza. La griglia `SPARSE` con 5% di ostacoli mostra un incremento di utilizzo in memoria del 66,5% (da 1230 a 2048 bytes) e una penalità prestazionale di 4,7x, rendendola già controproducente rispetto alle alternative. La griglia `DENSE` con 60% di ostacoli raggiunge un incremento del 1841,3% (quasi 19 volte più memoria), dimostrando la completa impraticabilità di questo approccio per scenari ad alta densità.

Questa limitazione ha guidato la decisione di includere `SparseGrid` nel sistema non come implementazione generale, ma come ottimizzazione specializzata per casi d'uso molto specifici dove la densità di ostacoli è garantita essere inferiore al 2-3% e dove la memoria è il requisito primario.

3.4 CompressedGrid

A differenza di `BitSetGrid` che delega la gestione bit-level a Java, `CompressedGrid` implementa direttamente le operazioni bitwise per occupare meno memoria ed evitare overhead introdotto da classi generiche come BitSet, sfruttando le operazioni che il processore esegue in modo molto efficiente.

La realizzazione utilizza un array di byte dove ogni bit rappresenta una cella, con operazioni bitwise esplicite per setting e getting. Ad esempio, per sapere se una cella è attraversabile, ho implementato un'operazione teoricamente molto veloce:
`(obstacles[index / 8] & (1 << (index % 8))) == 0.`

Nei risultati sperimentali si nota eccellente efficienza per la griglia SPARSE (con penalità 2.7x e risparmio memoria 84.9%), e `MAZE` (con penalità 1,5x e risparmio 72,9%). Tuttavia, la griglia `PATTERN` evidenzia che l'overhead diventa significativo (on penalità 4,0x) in scenari dove l'algoritmo di pathfinding termina rapidamente, rendendo le operazioni di accesso alla griglia proporzionalmente più costose. Infatti, in questi casi, il tempo speso nelle operazioni di accesso alla griglia diventa proporzionalmente dominante rispetto al tempo totale di esecuzione, rendendo l'opzione meno vantaggiosa.

4. Ottimizzazioni Temporali

4.1 Algoritmo FreePaths

L'algoritmo `FreePaths` rappresenta il cuore computazionale del sistema, e la sua ottimizzazione ha richiesto un'analisi dettagliata dei pattern di accesso alla memoria e delle allocazioni di oggetti.

La versione originale da me realizzata mostrava inefficienze significative dovute alla creazione ripetuta di oggetti `Cell` temporanei durante i calcoli di validità dei percorsi, che come già specificato portava a degradare le prestazioni. La riscrittura del metodo `calculateContextAndComplement()` elimina queste allocazioni utilizzando coordinate primitive per i controlli intermedi e creando oggetti `Cell` solo quando necessario per l'inserimento nelle strutture dati finali.

```
public void calculateContextAndComplement() {
    int oRow = origin.getRow();
    int oCol = origin.getCol();

    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {

            if (!grid.isTraversable(row, col) ||
            (row == oRow && col == oCol)) {
                continue;
            }

            if (hasValidFreePath(oRow, oCol, row, col, true)) {
                Cell cell = new Cell(row, col);
                context.add(cell);
                closure.add(new Landmark(cell, 1));
            } else if (hasValidFreePath(oRow, oCol, row, col, false)) {
                Cell cell = new Cell(row, col);
                complement.add(cell);
                closure.add(new Landmark(cell, 2));
            }
        }
    }
}
```

La separazione tra Type1 e Type2 nel calcolo dei cammini liberi non è solo una questione di correttezza algoritmica, ma rappresenta anche un'ottimizzazione per ridurre branch misprediction. Infatti i moderni processori cercano di indovinare in anticipo quali istruzioni verranno eseguite, per lavorare in parallelo e sfruttare al meglio la pipeline .Testare prima Type1 e poi Type2 in sequenza permette al branch predictor del processore di ottimizzare i pattern più comuni, migliorando il throughput complessivo anche se l’algoritmo rimane logicamente identico.

4.2 Euristica di Ordinamento della Frontiera

L'euristica di ordinamento della frontiera rappresenta l'ottimizzazione con il maggior impatto prestazionale dell'intero sistema. La decisione di ordinare i landmark della frontiera in base alla distanza libera dalla destinazione nasce dall'osservazione che algoritmi di pathfinding spendono la maggior parte del tempo esplorando direzioni subottimali prima di trovare il percorso corretto.

```
List<Landmark> sortedFrontier = new ArrayList<>(frontier);
sortedFrontier.sort((l1, l2) -> {
    double dist1 = freePathsCalculator.dLib(l1.getCell(), destination);
    double dist2 = freePathsCalculator.dLib(l2.getCell(), destination);
    return Double.compare(dist1, dist2);
}); // il metodo .sort() di ArrayList ha mediamente costo O(n log n)
```

L'implementazione utilizza una lambda expression ottimizzata che sfrutta il metodo `dLib()` già presente nel sistema per calcolare le distanze. Il costo O(n lgn) dell'ordinamento potrebbe sembrare eccessivo, ma l'analisi sperimentale dimostra che questo costo è ampiamente ammortizzato dalla riduzione drastica delle chiamate ricorsive che seguono.

La griglia `MAZE` mostra il miglioramento di uno speedup di 5,9x, passando da 15.959,652 ms a 2.721,624 ms, particolarmente significativo perché i labirinti rappresentano il worst case per algoritmi di pathfindinga causa dei molti vicoli ciechi. Il fatto che `ROOMS_AND_CORRIDORS` mostri uno speedup più modesto di 1,2x suggerisce che l'euristica è meno efficace in ambienti con strutture geometriche regolari dove la maggior parte delle direzioni porta comunque a soluzioni ragionevoli con opzioni di movimento molto più libere.

4.3 Condizione di Pruning

La formula `lF (==dl(O,F)) + lFD (==dl(F,D)) >= lunghezzaMin` rappresenta un lower bound sulla lunghezza di qualsiasi percorso che passi attraverso la cella corrente: se questa distanza è maggiore o uguale alla migliore soluzione trovata finora, esplorare quel ramo è matematicamente inutile.

```
if (lF + lFD >= lunghezzaMin) {
    iterationsFalse++;
    continue;
}
```

L'implementazione include un contatore `iterationsFalse` che permette di monitorare l'efficacia del pruning nella sperimentazione, visibile nei risultati come "Riga16 Falsi".

Un caso emblematico si verifica nella griglia `MAZE`, dove la sperimentazione mostra mediamente 550 iterazioni false su 858 frontiere totali per query, rappresentando un pruning rate del 64%. Questo significa che quasi due terzi delle potenziali esplorazioni

ricorsive vengono eliminate senza computazione, andando a giustificare gran parte dello speedup osservato.

4.4 Sistema di Caching

Il sistema di caching che ho implementato nella versione camminoMinOpt è progettato per massimizzare il riutilizzo computazionale specificamente nel contesto della sperimentazione, dove lo stesso MinimumPathCalculator viene riutilizzato per multiple query sulla stessa griglia. È importante notare che in un utilizzo tipico dell'algoritmo, dove si istanzia un calculator per una singola query origine-destinazione, il caching non apporta molti benefici poiché non ci sono sottoproblemi ripetuti da memorizzare. Il vantaggio emerge solo quando si eseguono multiple query con lo stesso calculator, permettendo di riutilizzare risultati di sottoproblemi comuni tra query diverse.

La Map `pathCache`, memorizza risultati completi di pathfinding indicizzati da una chiave composita che include origine, destinazione e hash dell'insieme di ostacoli (per garantire che risultati con diversi insiemi di ostacoli non vengano confusi). L'utilizzo di `ConcurrentHashMap` rappresenta una scelta per preparare il sistema a future estensioni parallelizzate dell'algoritmo senza richiedere modifiche architetturali significative.

```
private final Map<String, PathResult> pathCache = new
ConcurrentHashMap<>();
String key = origin + "->" + destination + "|" + obstacles.hashCode();
if (pathCache.containsKey(key))
      return pathCache.get(key);
```

L'efficacia del caching varia significativamente tra le tipologie di griglia considerate, come evidenziato dal numero medio di chiamate ricorsive per query. La griglia `MAZE` con 185 chiamate medie rappresenta il caso con maggior potenziale di riutilizzo, dove sottoproblemi identici si ripresentano frequentemente durante l'esplorazione dei vicoli ciechi. `ROOMS_AND_CORRIDORS` con 17 chiamate medie mostra benefici moderati grazie alla struttura geometrica regolare che genera pattern ripetitivi.

5. Risultati Sperimentali

5.1 Validazione della Correttezza

Il metodo `isCorrect()` implementa una strategia di confronto robusta che gestisce sia i casi di percorsi finiti che quelli di destinazioni irraggiungibili. Per i percorsi finiti, il confronto utilizza una tolleranza di 1e-6 per gestire errori di rappresentazione floating-point (si considera sqrt(2) nella distanza libera), mentre per le destinazioni irraggiungibili verifica che entrambi i risultati restituiscano `Double.POSITIVE_INFINITY`.

```
public boolean isCorrect(PathResult result1, PathResult result2) {
    if (result1.getLength() == Double.POSITIVE_INFINITY &&
        result2.getLength() == Double.POSITIVE_INFINITY) {
        return true;
    } else {
        double diff = Math.abs(result1.getLength() -
result2.getLength());
        return diff < 1e-6;
    }
}
```

Il risultato del 100% di correttezza su tutte le configurazioni sperimentali per ogni configurazione di griglia conferma che nessuna delle ottimizzazioni implementate compromette l'accuratezza dell'algoritmo. Questo risultato è particolarmente significativo considerando la complessità delle ottimizzazioni introdotte, che includono caching, pruning e riordinamento euristico.

5.2 Confronto tra Strutture Dati

Le proiezioni di scalabilità derivate dai dati sperimentali forniscono indicazioni precise per l'utilizzo del sistema in applicazioni reali. Per griglie di 5 milioni di celle, `ArrayGrid` richiede teoricamente 4882,8 kB di memoria, mentre `BitSet` e `Compressed` circa 610 kB, un risparmio dell'87,5%. Questa differenza diventa critica in applicazioni embedded o mobile dove la memoria disponibile è limitata.

Viceversa `SparseGrid` dimostra la sua completa impraticabilità per applicazioni di larga scala, richiedendo fino a 92 MB per 5 milioni di celle (rispetto ai 4.8 MB di `ArrayGrid`), confermando le considerazioni che prevedevano venga riservato a scenari molto specifici con densità garantita inferiore al 3%.

La scelta della struttura dati ottimale emerge quindi come una decisione che deve considerare non solo le prestazioni pure ma anche i vincoli di memoria e i pattern di utilizzo specifici dell'applicazione target. Posso concludere dicendo che per applicazioni che processano griglie superiori a 1000×1000, `BitSetGrid` o `CompressedGrid` diventano obbligatori, mentre per applicazioni con vincoli di prestazioni stringenti e memoria abbondante, è preferibile l'`ArrayGrid`.

5.3 Analisi per Categorie di Distanza

La classificazione automatica dei risultati per categoria di distanza libera, implementata nel `PerformanceAnalyzer`, rivela pattern comportamentali specifici che guidano la comprensione dell'efficacia delle ottimizzazioni. Le distanze libere vengono suddivise in tre gruppi principali: celle vicine, celle medie e celle lontane.

Le celle vicine, con distanza libera inferiore a 10, mostrano tempi tipicamente sotto il millisecondo, indicando che in questi casi l'algoritmo trova rapidamente soluzioni attraverso i casi 1 (contesto diretto) o 2 (complemento diretto) senza ricorsione significativa. I dati confermano questo comportamento: in mappe di tipo `RANDOM`, la media dei tempi si attesta attorno a 0,13 millisecondi su 19 campioni, mentre in scenari `PATTERN` si scende addirittura a 0,09 millisecondi con 6 campioni.

Le celle medie con distanza tra 10 e 25 rappresentano il regime dove le ottimizzazioni iniziano a mostrare benefici significativi. In `MAZE` la distanza raggiunge 1,97 ms, su un totale di 19 campioni; in contesti `DENSE`, pur con un numero più limitato di osservazioni (6 campioni), il tempo medio rimane contenuto a 2,49 millisecondi.

Le celle lontane con distanza superiore a 25 rappresentano il regime dove le ottimizzazioni raggiungono la massima efficacia, richiedendo ricorsione profonda con molte esplorazioni. La griglia `SPARSE` con 4,53 ms per 8 query lontane dimostra come anche scenari complessi rimangano gestibili con le ottimizzazioni appropriate.

5.4 Versioni di CamminoMin

Il confronto tra versioni base e ottimizzata rivela pattern di miglioramento che variano drasticamente tra tipologie di griglia, fornendo insight preziosi sui meccanismi sottostanti alle ottimizzazioni.

La griglia `MAZE` rappresenta il caso più significativo con uno speedup di 5,9, passando da quasi 16 secondi a meno di 3. Questo miglioramento riflette la natura patologica dei labirinti per algoritmi di ricerca non guidati: la versione base esplora sistematicamente tutti i vicoli ciechi, mentre la versione ottimizzata li evita attraverso l'euristica di ordinamento.

La griglia `DENSE` con speedup di 0,9x mostra un risultato inaspettato dove la versione ottimizzata risulta leggermente più lenta (10.112 ms contro 8.675 ms). Questo apparente paradosso si spiega considerando che in griglie molto dense con pochi percorsi possibili, l'overhead introdotto dalle ottimizzazioni (ordinamento della frontiera, controlli di pruning, gestione della cache) supera i benefici. In questi scenari, l'algoritmo base che esplora sistematicamente risulta più efficiente proprio perché ci sono pochissime alternative da esplorare.

La griglia `ROOMS_AND_CORRIDORS` con speedup di 1.2x dimostra benefici moderati ma consistenti, mentre `PATTERN` e `RANDOM` mostrano miglioramenti variabili a seconda della complessità specifica delle query testate.

6. Limiti

6.1 Analisi dei Casi Patologici

Le prove di esecuzione hanno identificato limiti computazionali fondamentali che persistono nonostante tutte le ottimizzazioni implementate. Il caso più estremo osservato nella griglia `DENSE` ha richiesto oltre 120 secondi per una singola query, dimostrando che esistono configurazioni dove l'algoritmo mantiene complessità esponenziale. In questi scenari, l'algoritmo deve esplorare quasi tutti i possibili percorsi alternativi prima di identificare la soluzione ottimale, rendendo inefficaci anche le ottimizzazioni più sofisticate. Per questo motivo, ho escluso deliberatamente queste configurazioni estreme dal sistema di sperimentazione automatizzato, poiché l'analisi completa di tutte le coppie origine-destinazione avrebbe richiesto tempi computazionali proibitivi, compromettendo la fattibilità dell'intero processo di benchmarking."

L'identificazione di questi limiti dovrebbe portare alla definizione di soglie per l'utilizzo del sistema in applicazioni reali: per griglie con densità superiore al 50% e dimensioni oltre 40x40 con determinate configurazioni, è necessario implementare timeout per evitare che singole query monopolizzino risorse computazionali.

6.2 Limitazioni Architetturali: Stack e Memoria

L'implementazione ricorsiva nativa dell'algoritmo introduce limitazioni architetturali significative legate alla profondità massima dello stack. Durante i test più profondi, sono stati osservati stack overflow in scenari dove la ricorsione supera i 1000 livelli, tipicamente in griglie molto ampie con percorsi estremamente tortuosi. Questa limitazione richiederebbe configurazione manuale della JVM per aumentare la dimensione dello stack.

Allo stesso tempo, il sistema di caching introduce un potenziale memory leak: se le strutture utilizzate per memorizzare i risultati (come la pathCache) non vengono svuotate o gestite con meccanismi di invalidazione o limiti di dimensione, esse tendono a crescere progressivamente con ogni nuova query. In scenari a lungo termine, o in applicazioni embedded o server-based con uptime prolungato, questo comportamento può portare a un accumulo incontrollato di dati in memoria, causando un degrado delle prestazioni fino ad arrivare, nei casi più gravi, a un esaurimento della memoria disponibile (OutOfMemoryError in Java).